FuelWise — Documentação de Arquitetura de Software

arc42 · C4 Model (Níveis 1–4) · UML · Diagrama de Entidade-Relacionamento · Software Architecture Canvas

Equipe: Lucas A. Theiss, Artur Ribeiro de Souza, Cristhian Rezende, Vinicius da Rosa Padilha

Setembro de 2026

Sumário

Software Architecture Canvas **3**

1. Introdução e Objetivos **4**
1.1 Visão Geral dos Requisitos 4
1.2 Objetivos de Qualidade 5
1.2.1 Rastreabilidade entre requisitos e qualidade 5
1.3 Partes Interessadas 5

2. Restrições Arquiteturais **7**
2.1 Restrições Técnicas 7
2.2 Restrições Organizacionais 7

3. Contexto e Escopo **8**
3.1 Contexto Negocial 8
3.2 Contexto Técnico 9

4. Estratégia de Solução **10**

5. Visão de Blocos de Construção **11**
5.1 Nível 1 – Visão Sistêmica Geral (Caixa-branca do sistema) 11
5.2 Nível 2 – Decomposição da API (Caixa-branca) 12
5.3 Nível 3 – Pagamento do Abastecimento 13
5.4 Nível 3 – Cadastro e Aprovação de Transportadora 14
5.5 Nível 4 – Modelo de Domínio (UML de Classes) 15
5.6 Nível 4 – Componentes de Código do Módulo de Pagamento (UML de Componentes) 16

6. Visão de Tempo de Execução **18**
6.1 Cenário – Pagamento do Abastecimento 18
6.2 Cenário – Aprovação de Cadastro de Transportadora 18

7. Visão de Implantação **20**
7.1 Nível de Infraestrutura 1 20
7.2 Ambientes 20

8. Conceitos Transversais **22**
8.1 Segurança 22
8.2 Comunicação 22
8.3 Persistência 22

8.4 Notificações 24
8.5 Tratamento de Erros e Resiliência 24
8.6 Observabilidade 25

9. Decisões Arquiteturais **26**

10. Requisitos de Qualidade **28**
10.1 Árvore de Qualidade 28
10.2 Cenários de Qualidade 28

11. Riscos e Débitos Técnicos **30**

12. Glossário **31**

Software Architecture Canvas

Modelo: Software Architecture Canvas (workingsoftware.dev). Cada bloco é mapeado diretamente às seções do arc42 apresentado a seguir.

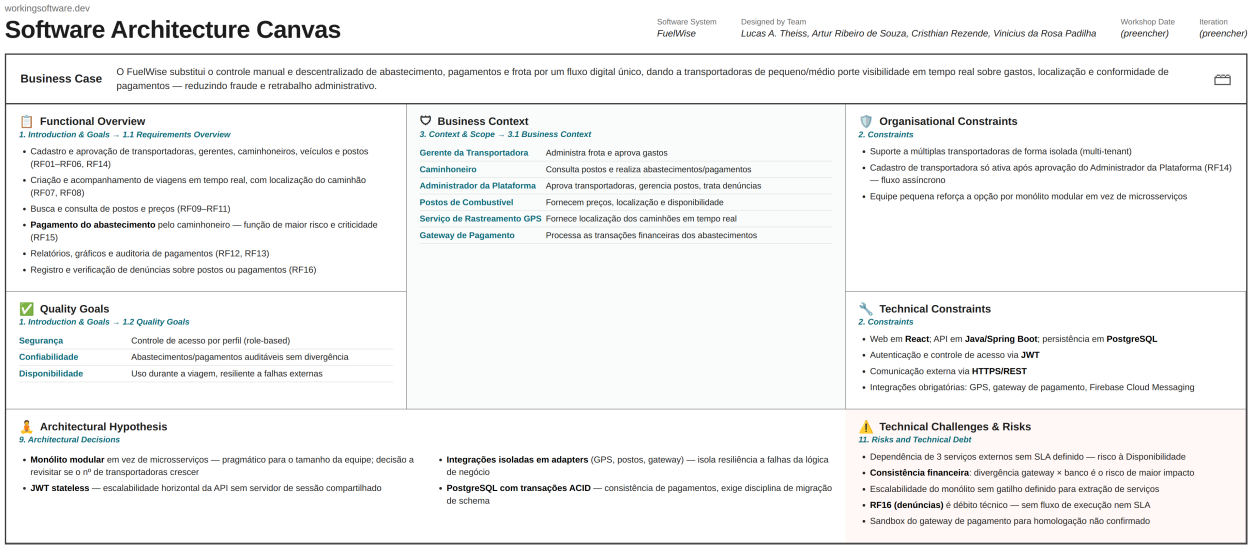


Figura 1: Software Architecture Canvas — FuelWise

1. Introdução e Objetivos

1.1 Visão Geral dos Requisitos

O FuelWise é uma plataforma web destinada a transportadoras de pequeno e médio porte, com foco na centralização do controle de combustível, pagamentos e gestão da frota.

O sistema busca substituir processos manuais e descentralizados por um fluxo integrado de cadastro, autorização, abastecimento, pagamento e acompanhamento da frota.

Os principais requisitos funcionais são:

Código	Requisito
RF01	Cadastrar transportadora na plataforma
RF02	Cadastrar gerente da transportadora
RF03	Cadastrar caminhoneiros
RF04	Cadastrar veículos da frota
RF05	Cadastrar postos de combustível
RF06	Manter tabela de preços dos combustíveis por posto
RF07	Relacionar caminhoneiro a veículo, criando uma viagem
RF08	Acompanhar em tempo real a localização do caminhão e do caminhoneiro
RF09	Localizar postos de combustível disponíveis
RF10	Visualizar dados dos postos
RF11	Acessar painel com postos filiados
RF12	Gerar relatórios e gráficos de abastecimento
RF13	Auditar dados de pagamentos das viagens
RF14	Aprovar ou recusar solicitações de cadastro de transportadora
RF15	Realizar pagamento de abastecimento pelo caminhoneiro
RF16	Registrar e verificar denúncias sobre postos ou pagamentos

> **Correção:** o RF07 usava o termo “funcionário”, inconsistente com o restante do documento. Foi padronizado para “caminhoneiro”, único perfil operacional que executa viagens.

1.2 Objetivos de Qualidade

Prioridade	Objetivo	Justificativa
Alta	Segurança	O sistema manipula dados de usuários, viagens e pagamentos e precisa controlar o acesso às operações por perfil.
Alta	Confiabilidade	Registros de abastecimento e pagamentos devem ser persistidos corretamente para permitir auditoria (RF13).
Alta	Disponibilidade	O caminhoneiro precisa acessar o sistema durante a viagem para consultar postos e realizar abastecimentos, mesmo com falhas pontuais em serviços externos.
Média	Desempenho	Consultas de postos, localização e operações de abastecimento devem apresentar resposta adequada ao uso operacional.
Média	Manutenibilidade	A separação entre aplicação web, API, serviços e banco facilita a evolução do sistema.

1.2.1 Rastreabilidade entre requisitos e qualidade

Objetivo de qualidade	Requisitos funcionais relacionados
Segurança	RF01, RF02, RF03, RF14 (cadastro/aprovação controlados por perfil)
Confiabilidade	RF13, RF15, RF16 (pagamento, auditoria, denúncia)
Disponibilidade	RF08, RF09, RF15 (uso durante a viagem)
Desempenho	RF09, RF10, RF11 (consultas de postos)
Manutenibilidade	Todos — decorre da separação em componentes descrita na Seção 5

1.3 Partes Interessadas

Parte interessada	Expectativas
Gerente da Transportadora	Administrar frota e caminhoneiros, acompanhar a localização e controlar/aprovar gastos.
Caminhoneiro	Consultar postos, realizar abastecimentos e pagamentos com agilidade durante a viagem.

Parte interessada	Expectativas
Administrador da Plataforma	Manter postos e transportadoras cadastrados e confiáveis, e tratar denúncias (RF16).
Equipe de Desenvolvimento	Ter uma arquitetura organizada, modular e adequada à evolução do sistema.

2. Restrições Arquiteturais

2.1 Restrições Técnicas

- Aplicação web utilizando **React** como interface do usuário.
- API desenvolvida em **Java com Spring Boot**.
- Persistência utilizando **PostgreSQL**.
- Autenticação e controle de acesso utilizando **JWT**.
- Comunicação entre aplicações e serviços externos utilizando **HTTPS e APIs REST**.
- Utilização de um serviço externo de rastreamento GPS para localização dos veículos.
- Utilização de um gateway externo para processamento dos pagamentos.
- Utilização de Firebase Cloud Messaging para envio de notificações.

2.2 Restrições Organizacionais

- O sistema deve suportar múltiplas transportadoras de forma isolada (dados de uma transportadora não podem ser acessados por outra).
- O cadastro de uma transportadora só se torna ativo após aprovação do Administrador da Plataforma (RF14), o que introduz um fluxo assíncrono de ativação.
- A equipe é pequena, o que reforça a necessidade de uma arquitetura simples de operar (monólito modular via Spring Boot), evitando o custo operacional de microsserviços nesta fase.

3. Contexto e Escopo

3.1 Contexto Negocial

O FuelWise atua como elemento central entre os usuários da transportadora e serviços externos relacionados ao abastecimento.

Elemento externo	Interação com o FuelWise
Gerente da Transportadora	Administra frota, caminhoneiros e aprova pagamentos.
Caminhoneiro	Consulta postos e realiza abastecimentos.
Administrador da Plataforma	Aprova transportadoras, gerencia postos e trata denúncias.
Postos de Combustível	Disponibilizam informações de localização, preços e abastecimento.
Serviço de Rastreamento GPS	Fornece a localização dos caminhões em tempo real.
Gateway de Pagamento	Processa os pagamentos dos abastecimentos.

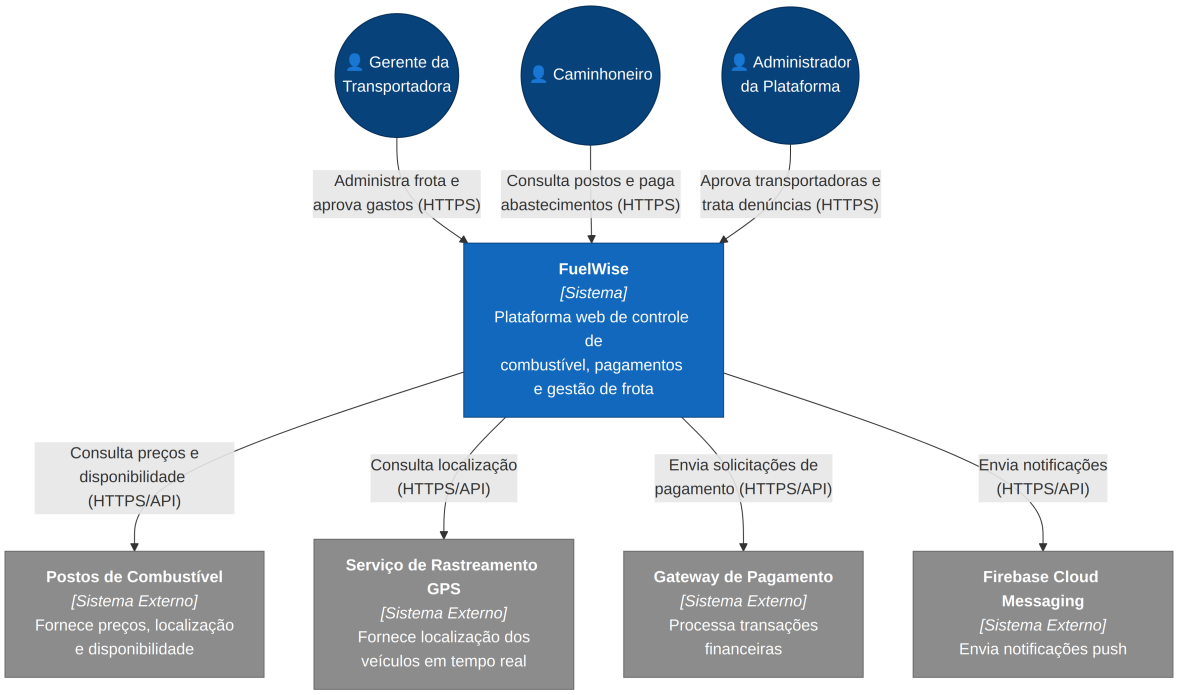


Figura 2: Diagrama de Contexto do Sistema (C4 Nível 1)

3.2 Contexto Técnico

As principais integrações técnicas são realizadas por APIs utilizando HTTPS.

- A aplicação web se comunica com a API do FuelWise por HTTPS/JSON.
- A API consulta informações de postos por HTTPS/API.
- A API consulta o serviço GPS por HTTPS/API.
- A API comunica-se com o gateway de pagamento por HTTPS/API.
- A autenticação utiliza JWT, emitido pela API e validado a cada requisição.
- O acesso ao banco PostgreSQL é realizado exclusivamente pela API (nenhum outro componente acessa o banco diretamente).
- O serviço de notificações utiliza Firebase para enviar confirmações e alertas ao caminhoneiro.

4. Estratégia de Solução

A arquitetura do FuelWise é organizada como um **monólito modular**: uma aplicação web, uma API central e serviços externos integrados.

A aplicação web, desenvolvida em React, fornece as interfaces utilizadas pelos três perfis de usuário (Gerente, Caminhoneiro, Administrador). A API Java/Spring Boot concentra as regras de negócio e as operações do sistema, organizada internamente em módulos por domínio (Cadastro, Frota, Viagem, Pagamento, Postos, Denúncia).

O PostgreSQL é utilizado para persistir os dados de usuários, veículos, viagens, abastecimentos, pagamentos e denúncias.

As integrações externas são isoladas em componentes de integração dentro da API, permitindo que o sistema consulte postos, obtenha a localização dos veículos, processe pagamentos e envie notificações sem acoplar diretamente essas responsabilidades à interface nem à lógica de negócio (ver Seção 8, Tratamento de Erros e Resiliência).

A autenticação utiliza JWT com claims de perfil (role) para controlar o acesso às funcionalidades de acordo com o usuário autenticado.

5. Visão de Blocos de Construção

5.1 Nível 1 – Visão Sistêmica Geral (Caixa-branca do sistema)

O sistema pode ser dividido nos seguintes blocos principais:

Bloco	Responsabilidade
Aplicação Web	Interface utilizada pelos usuários do FuelWise.
Autenticação	Realizar login e controlar o acesso utilizando JWT.
API	Processar as operações e regras de negócio do sistema.
Banco de Dados	Armazenar dados de usuários, veículos, viagens, abastecimentos, pagamentos e denúncias.
Notificações	Enviar confirmações e alertas aos usuários.
Integração com Postos	Consultar preços, localização e disponibilidade dos postos.
Integração GPS	Obter a localização dos caminhões.
Integração de Pagamento	Comunicar-se com o gateway para processar abastecimentos.

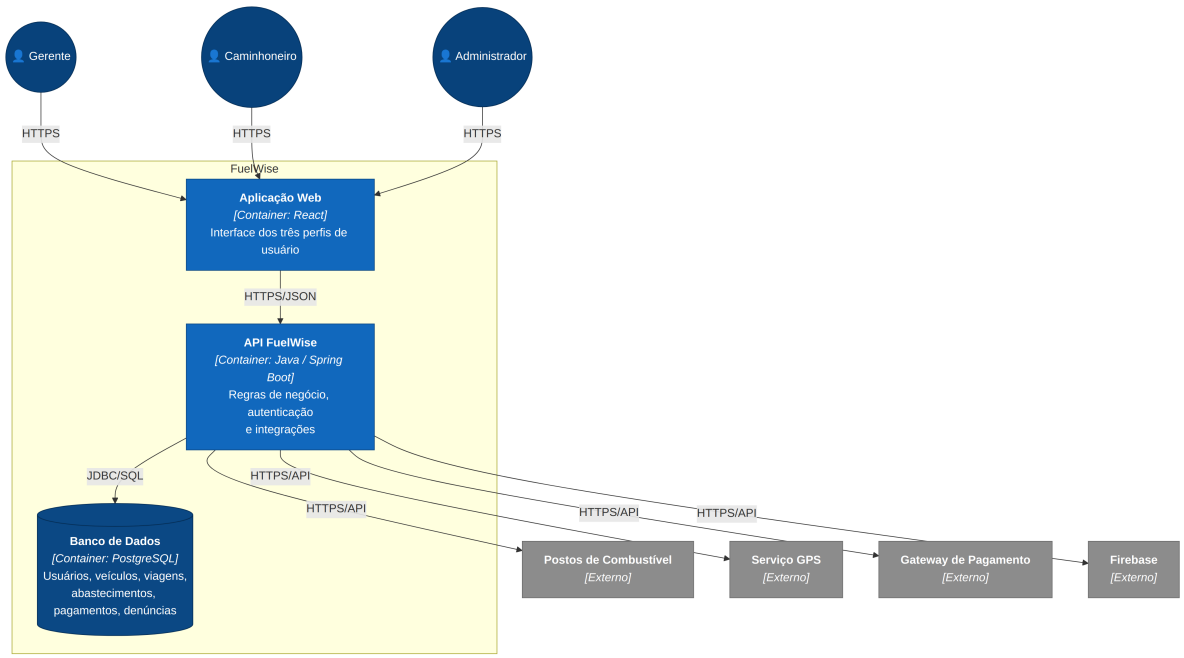


Figura 3: Diagrama de Containers (C4 Nível 2)

5.2 Nível 2 – Decomposição da API (Caixa-branca)

A API é decomposta nos seguintes módulos internos, cada um responsável por um domínio de negócio:

Módulo	Responsabilidade	Requisitos atendidos
Módulo de Autenticação e Acesso	Login, emissão/validação de JWT, controle de perfil (role-based access).	Transversal
Módulo de Cadastro	Cadastro e aprovação de transportadoras, gerentes, caminhoneiros, veículos e postos.	RF01–RF06, RF14
Módulo de Frota e Viagem	Relaciona caminhoneiro e veículo, cria e acompanha viagens.	RF07, RF08
Módulo de Postos	Consulta de postos, preços e disponibilidade, painel de filiados.	RF09, RF10, RF11
Módulo de Pagamento	Coordena a autorização e o pagamento de abastecimentos (detalhado na Seção 5.3).	RF15
Módulo de Relatórios e Auditoria	Gera relatórios/gráficos de abastecimento e audita pagamentos.	RF12, RF13

Módulo	Responsabilidade	Requisitos atendidos
Módulo de Denúncias	Registra e permite a verificação de denúncias sobre postos ou pagamentos por um administrador.	RF16
Módulo de Notificações	Dispara eventos de notificação (Firebase) a partir de ações dos demais módulos.	Transversal

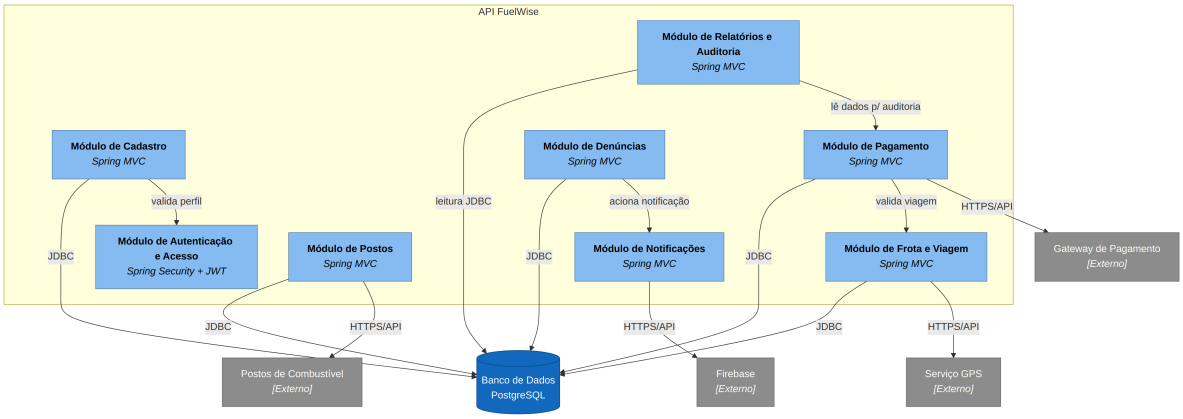


Figura 4: Diagrama de Componentes da API por Módulo (C4 Nível 3 — visão geral)

5.3 Nível 3 – Pagamento do Abastecimento

O módulo de Pagamento é dividido nos seguintes componentes:

Componente	Responsabilidade
Controlador de Pagamento	Receber a solicitação de pagamento vindo da aplicação web.
Serviço de Pagamento	Validar e coordenar o processo de pagamento.
Validador de Abastecimento	Verificar viagem, valor e autorização do caminhoneiro.
Integração de Pagamento	Comunicar-se com o gateway de pagamento externo.
Repositório de Pagamento	Persistir o resultado da transação no PostgreSQL.
Serviço de Notificação	Solicitar a confirmação do pagamento ao caminhoneiro.

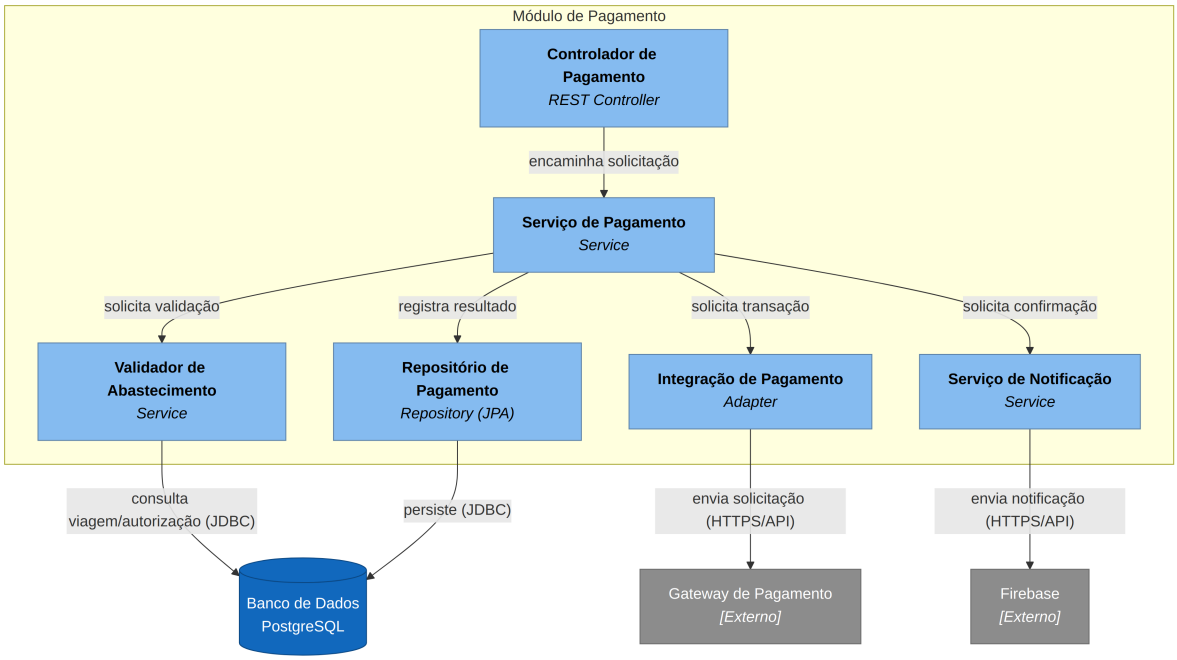


Figura 5: Diagrama de Componentes do Módulo de Pagamento (C4 Nível 3)

5.4 Nível 3 – Cadastro e Aprovação de Transportadora

Módulo relevante por concentrar o fluxo de ativação de novos clientes na plataforma (RF01, RF14):

Componente	Responsabilidade
Controlador de Cadastro	Recebe a solicitação de cadastro de transportadora.
Serviço de Cadastro	Valida os dados e registra a transportadora com status “pendente”.
Repositório de Transportadora	Persiste os dados cadastrais.
Painel de Aprovação (Administrador)	Lista solicitações pendentes para o Administrador da Plataforma.
Serviço de Aprovação	Aplica a decisão do Administrador (aprovar/recusar) e altera o status da transportadora.
Serviço de Notificação	Informa ao Gerente o resultado da aprovação.

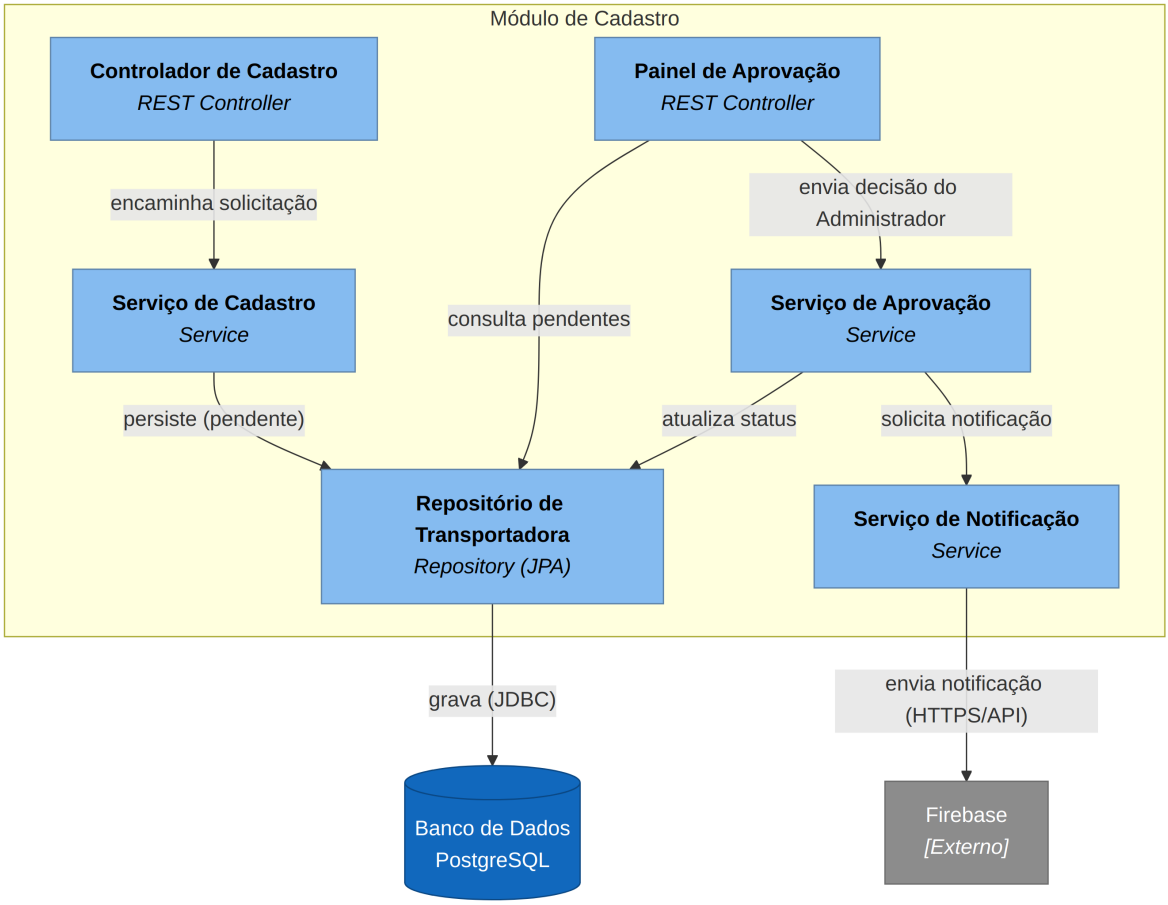


Figura 6: Diagrama de Componentes do Módulo de Cadastro (C4 Nível 3)

5.5 Nível 4 – Modelo de Domínio (UML de Classes)

Modelo de domínio derivado das entidades citadas no Glossário (Seção 12) e nos módulos das Seções 5.2–5.4, detalhando atributos, operações e multiplicidades entre as entidades do sistema.

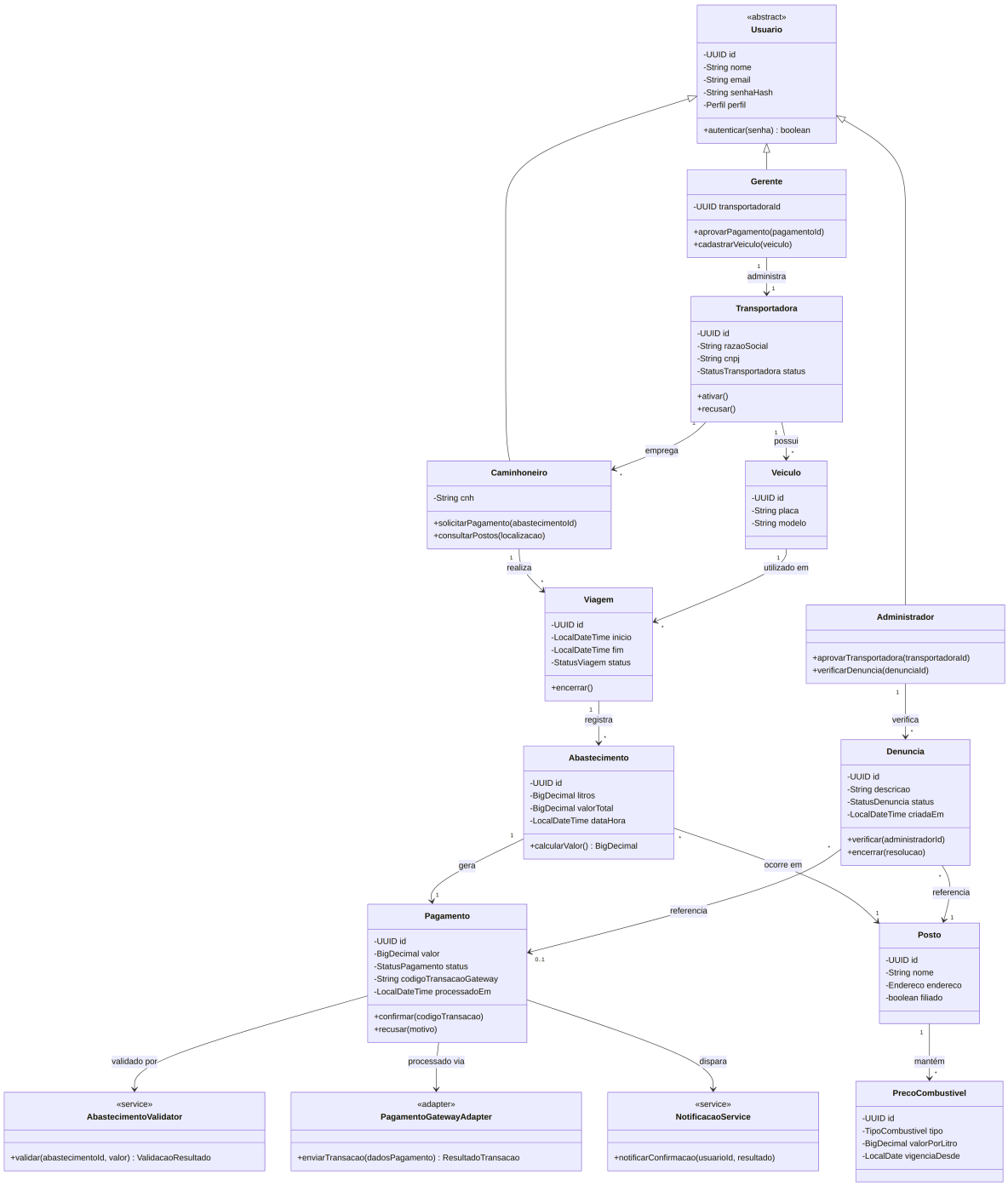


Figura 7: UML de Classes — Domínio de Pagamento e Entidades Relacionadas (C4 Nível 4)

5.6 Nível 4 – Componentes de Código do Módulo de Pagamento (UML de Componentes)

Detalha as classes/interfaces do módulo de Pagamento organizadas por camada (Controller → Service → Repository/Adapter), incluindo as interfaces expostas e requeridas entre elas — refinamento em nível de

código da Figura 5.3.

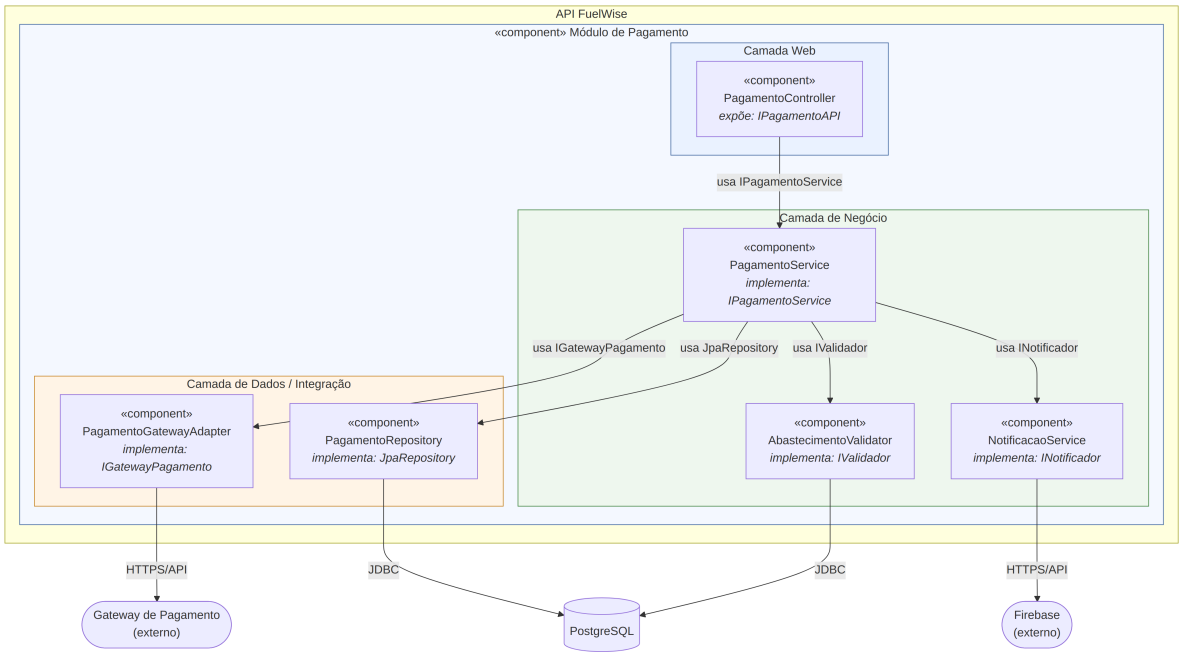


Figura 8: UML de Componentes — Módulo de Pagamento em nível de código (C4 Nível 4)

Interfaces expostas/requeridas do módulo:

Interface	Fornecida por	Consumida por	Descrição
`IPagamentoAPI`	PagamentoController	Aplicação Web (via HTTP)	Contrato REST para solicitar pagamento
`IPagamentoService`	PagamentoService	PagamentoController	Orquestra a validação, integração e persistência
`IValidador`	AbastecimentoValidator	PagamentoService	Verifica viagem, valor e autorização
`IGatewayPagamento`	PagamentoGatewayAdapter	PagamentoService	Abstrai o protocolo do gateway externo
`INotificador`	NotificacaoService	PagamentoService	Abstrai o canal de notificação (Firebase)
`JpaRepository<Pagamento>`	PagamentoRepository	PagamentoService	Persistência do agregado Pagamento

6. Visão de Tempo de Execução

6.1 Cenário – Pagamento do Abastecimento

1. O caminhoneiro solicita o pagamento pela aplicação web.
2. A aplicação envia a solicitação para o Controlador de Pagamento da API.
3. O controlador encaminha a solicitação ao Serviço de Pagamento.
4. O Serviço de Pagamento solicita ao Validador a verificação da viagem, valor e autorização.
5. O Validador consulta os dados necessários no banco PostgreSQL.
6. Após a validação, o Serviço de Pagamento solicita a transação à Integração de Pagamento.
7. A Integração de Pagamento envia a solicitação ao gateway externo.
8. O gateway retorna o resultado da transação.
9. O Serviço de Pagamento registra o resultado através do Repositório de Pagamento.
10. O Serviço de Notificação solicita o envio da confirmação.
11. A confirmação é enviada ao caminhoneiro via Firebase.

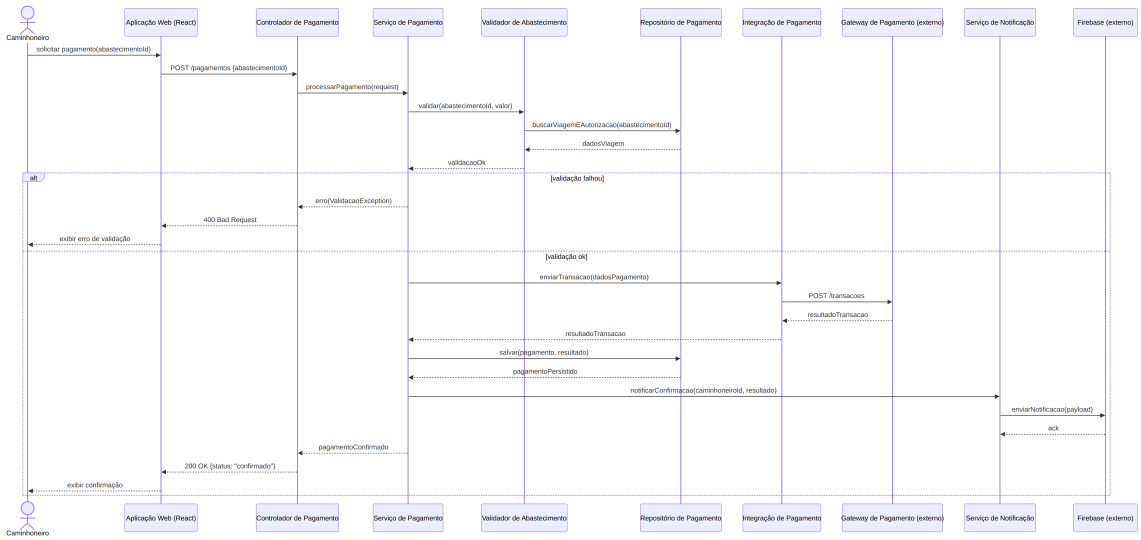


Figura 9: UML de Sequência — Pagamento do Abastecimento (C4 Nível 4)

6.2 Cenário – Aprovação de Cadastro de Transportadora

1. O Gerente submete o cadastro da transportadora pela aplicação web.
2. A aplicação envia os dados ao Controlador de Cadastro da API.

3. O Serviço de Cadastro valida os dados e persiste a transportadora com status “pendente” via Repositório de Transportadora.
4. O Administrador da Plataforma consulta o Painel de Aprovação e visualiza a solicitação.
5. O Administrador aprova ou recusa a solicitação.
6. O Serviço de Aprovação atualiza o status da transportadora no banco.
7. O Serviço de Notificação envia o resultado ao Gerente via Firebase.
8. Caso aprovada, o Gerente passa a ter acesso liberado às funcionalidades de cadastro de frota e caminhoneiros.

Nota: apenas dois cenários eram documentados na versão original (havia apenas um). Recomenda-se, em uma futura revisão, adicionar também o cenário de “Registro de Denúncia” (RF16), por envolver moderação e possível suspensão de posto/transportadora — um fluxo com impacto direto na Confiabilidade do sistema.

7. Visão de Implantação

7.1 Nível de Infraestrutura 1

A implantação prevista utiliza uma arquitetura distribuída em nuvem, composta pela aplicação web, API, banco de dados e serviços externos.

Elemento	Responsabilidade	Observações de implantação
CDN / Hospedagem estática	Serve os artefatos compilados do React.	Ex.: bucket + CDN (S3/CloudFront ou equivalente).
Servidor da API	Executa a aplicação Java/Spring Boot, empacotada em contêiner (Docker).	Executado atrás de um load balancer, com múltiplas instâncias para atender ao objetivo de Disponibilidade.
Banco PostgreSQL	Persiste os dados do sistema.	Instância gerenciada, com backup automatizado, em ambiente separado de aplicação.
Serviço GPS	Fornece localização dos veículos.	Externo, consumido via HTTPS/API.
Gateway de Pagamento	Processa transações financeiras.	Externo, consumido via HTTPS/API.
Firebase	Envia notificações aos usuários.	Externo, consumido via HTTPS/API.
Serviços de Postos	Disponibilizam informações de preços, localização e abastecimento.	Externo, consumido via HTTPS/API.

7.2 Ambientes

Ambiente	Finalidade
Desenvolvimento	Testes locais e integração contínua.
Homologação	Validação funcional antes da produção, com integrações externas em modo sandbox (especialmente gateway de pagamento).
Produção	Ambiente ativo, com múltiplas instâncias da API atrás de load balancer para suportar picos de uso.

A comunicação entre a aplicação e a API, bem como entre a API e os serviços externos, ocorre por APIs utilizando HTTPS em todos os ambientes.

8. Conceitos Transversais

8.1 Segurança

A autenticação dos usuários utiliza JWT. O sistema deve controlar o acesso às funcionalidades de acordo com o perfil do usuário (role-based access control), diferenciando Gerente, Caminhoneiro e Administrador. Tokens possuem tempo de expiração e devem ser renovados via refresh token.

8.2 Comunicação

As integrações entre os componentes e serviços externos utilizam APIs REST sobre HTTPS. A comunicação entre a aplicação web e a API ocorre por HTTPS/JSON.

8.3 Persistência

O PostgreSQL é utilizado como banco de dados principal. A API é a única responsável pelas operações de persistência e consulta das informações utilizadas pelo sistema.

Decisões de modelagem do DER:

- Como o sistema tem três perfis de usuário com atributos distintos (Gerente, Caminhoneiro, Administrador), optou-se por **tabela por subtipo**: `usuario` guarda os dados comuns de autenticação e cada perfil tem sua própria tabela ligada por chave estrangeira 1:1 a `usuario`. Isso evita colunas nulas em excesso (problema comum de "tabela única com discriminador" quando os subtipos têm poucos atributos em comum).
- `pagamento` tem relação 1:1 com `abastecimento` (cada abastecimento gera no máximo um pagamento), refletindo o fluxo descrito na Seção 6.1.
- `denuncia` tem chaves estrangeiras **opcionais** para `posto` e `pagamento`, pois o RF16 permite denunciar tanto um posto quanto um pagamento — nunca os dois ao mesmo tempo nem nenhum dos dois isoladamente sem contexto (essa regra de negócio deve ser garantida em aplicação, não é representável apenas pelo DER).
- `preco_combustivel` guarda histórico de vigência (`vigencia_desde`) em vez de atualizar o preço em uma única linha, para permitir auditoria de preços ao longo do tempo (RF06, RF12).

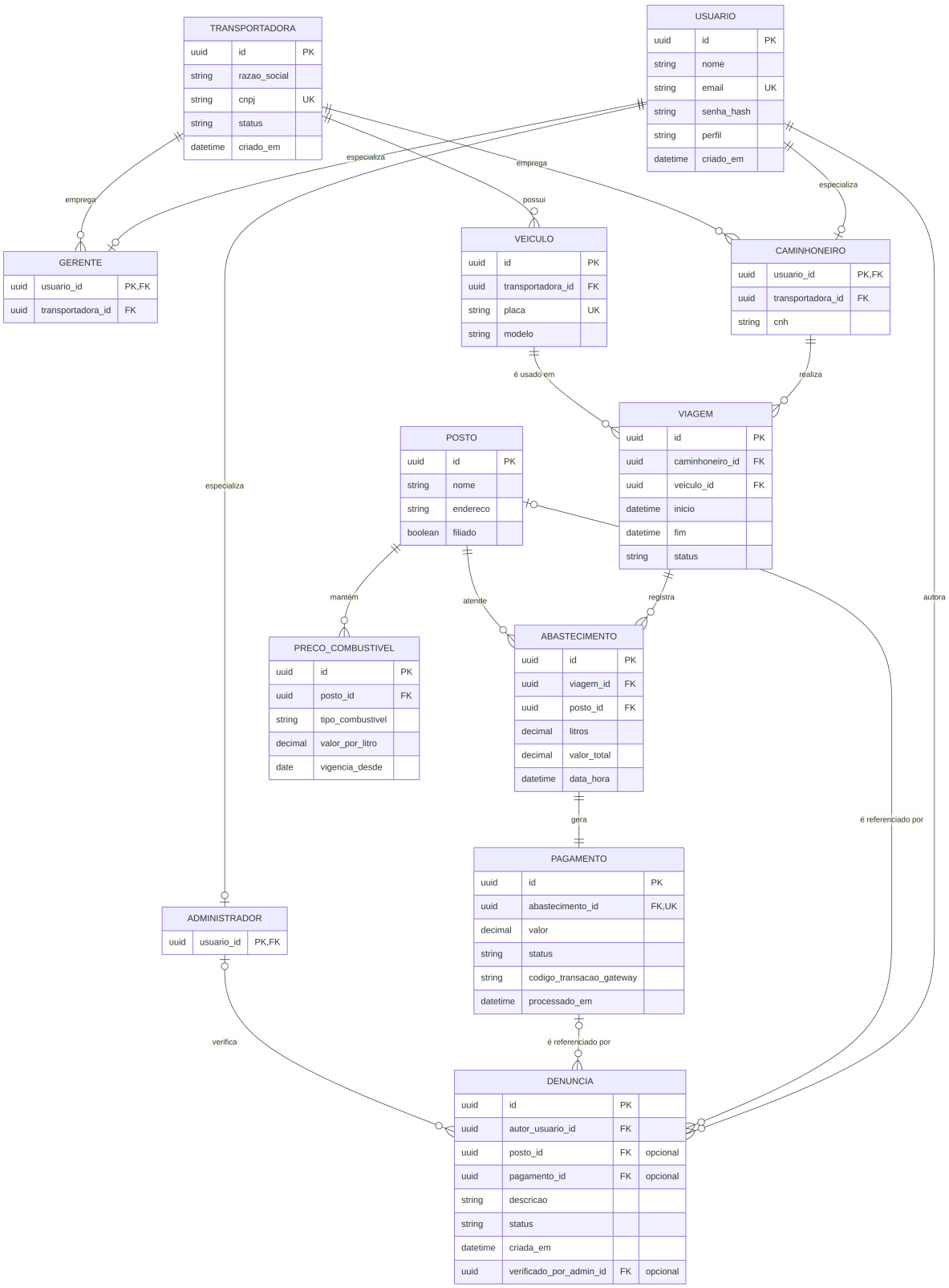


Figura 10: Diagrama de Entidade-Relacionamento (DER)

Dicionário de dados resumido:

Tabela	Propósito	Observação de integridade
`usuario`	Dados comuns de autenticação de todos os perfis.	`email` único; `perfil` restringe o subtipo esperado.
`gerente` / `caminhoneiro` / `administrador`	Atributos específicos de cada perfil.	Chave primária é também chave estrangeira para `usuario` (relação 1:1).
`transportadora`	Empresa cliente da plataforma.	`status` controla o fluxo de aprovação (RF14): `pendente`, `ativa`, `recusada`.
`veiculo`	Veículo da frota de uma transportadora.	`placa` única no sistema.
`viagem`	Associação entre caminhoneiro e veículo (RF07).	Um caminhoneiro pode ter várias viagens ao longo do tempo, mas não duas viagens simultaneamente "em andamento" — regra de aplicação.
`posto`	Posto de combustível cadastrado/consultável.	`filiado` diferencia postos parceiros dos apenas listados (RF09–RF11).
`preco_combustivel`	Histórico de preços por posto e tipo de combustível.	Mantém série histórica via `vigencia_desde`, não sobrescreve.
`abastecimento`	Operação de compra de combustível.	Sempre vinculado a uma viagem e a um posto.
`pagamento`	Resultado financeiro de um abastecimento.	`abastecimento_id` é único — no máximo um pagamento por abastecimento.
`denuncia`	Registro de irregularidade (RF16).	Referencia posto **ou** pagamento (nunca os dois), validado em aplicação.

8.4 Notificações

As confirmações e alertas destinados ao caminhoneiro (e, quando aplicável, ao gerente) são enviados por meio de um serviço de notificações baseado em Firebase Cloud Messaging.

8.5 Tratamento de Erros e Resiliência

Como o sistema depende de três serviços externos (GPS, postos, gateway de pagamento) considerados risco na Seção 11, cada integração deve:

- Aplicar timeout e retry com backoff em chamadas externas.
- Isolar falhas por integração (uma falha na integração de postos não deve impedir consultas de GPS ou pagamentos).
- Registrar (log) falhas de integração para acompanhamento operacional.

- Retornar mensagens de erro adequadas à aplicação web quando um serviço externo estiver indisponível, evitando estados inconsistentes (especialmente em pagamentos).

8.6 Observabilidade

A API deve expor logs estruturados das operações críticas (pagamento, aprovação de cadastro, denúncias) para permitir auditoria (RF13) e apoiar a investigação de incidentes.

9. Decisões Arquiteturais

Decisão	Alternativas consideradas	Justificativa da escolha	Consequências
React no frontend	Angular, Vue.js	Ecossistema maduro e ampla disponibilidade de desenvolvedores; separa claramente UI da lógica de negócio da API.	Necessita de uma API bem definida (contrato REST) desde o início.
Java/Spring Boot na API	Node.js/NestJS, .NET	Robustez para regras de negócio complexas (pagamento, auditoria) e maturidade do ecossistema Spring para segurança e integrações.	Maior verbosidade de código; exige times com experiência em Java.
PostgreSQL	MySQL, MongoDB	Suporte forte a transações ACID, essencial para consistência de pagamentos e auditoria.	Modelagem relacional exige migrações controladas de schema.
JWT para autenticação	Sessão com cookie + servidor de sessão	Stateless, adequado a múltiplos clientes (web e futura aplicação mobile) e a escalabilidade horizontal da API.	Necessita de estratégia de revogação/expiração de token.
Monólito modular (em vez de microsserviços)	Arquitetura de microsserviços por domínio	Equipe pequena e escopo inicial não justificam o custo operacional de microsserviços; módulos internos preservam a possibilidade de extração futura.	Deploy único da API; times devem manter disciplina de baixo acoplamento entre módulos.

Decisão	Alternativas consideradas	Justificativa da escolha	Consequências
Integrações externas isoladas em componentes dedicados	Chamadas diretas dos serviços de domínio às APIs externas	Isola a lógica de negócio de detalhes de integração, facilitando troca de fornecedor (ex.: gateway de pagamento) e tratamento de falhas.	Camada adicional de abstração a manter.

10. Requisitos de Qualidade

10.1 Árvore de Qualidade

- **Qualidade**
  - Segurança
    - Autenticação dos usuários
    - Controle de acesso por perfil
  - Confiabilidade
    - Persistência dos abastecimentos
    - Registro dos pagamentos
    - Auditoria das transações
    - Registro de denúncias
  - Disponibilidade
    - Acesso ao sistema durante as viagens
    - Disponibilidade das consultas de postos
    - Resiliência a falhas de serviços externos
  - Desempenho
    - Consultas de postos
    - Operações de abastecimento
  - Manutenibilidade
    - Separação entre frontend, API, banco e integrações
    - Organização da API em módulos por domínio

10.2 Cenários de Qualidade

Qualidade	Cenário	Métrica sugerida
Segurança	Um usuário deve acessar somente as funcionalidades permitidas ao seu perfil.	100% das rotas protegidas por verificação de role.
Confiabilidade	Após um pagamento, o resultado da transação deve ser registrado no banco de dados.	Nenhuma transação de pagamento sem registro correspondente.

Qualidade	Cenário	Métrica sugerida
Disponibilidade	O caminhoneiro deve conseguir consultar postos e realizar operações durante uma viagem quando os serviços necessários estiverem disponíveis.	Disponibilidade mensal da API ≥ 99%.
Desempenho	Consultas de postos e operações comuns devem retornar sem atrasos que prejudiquem o uso durante a viagem.	Tempo de resposta ≤ 2s em 95% das requisições.
Manutenibilidade	Alterações em uma integração externa devem ser realizadas preferencialmente no componente responsável por essa integração, sem necessidade de alterar a interface do usuário.	Mudança de fornecedor de integração sem alteração no frontend.

11. Riscos e Débitos Técnicos

Risco	Impacto	Mitigação
Dependência de serviços externos	Indisponibilidade de GPS, pagamento, postos ou notificações pode afetar funcionalidades do sistema.	Isolar integrações em componentes específicos, aplicar timeout/retry e tratar falhas de comunicação (ver Seção 8.5).
Segurança de pagamentos	Falhas no processo de autenticação ou autorização podem gerar transações indevidas.	Utilizar autenticação, autorização e validações antes do processamento.
Consistência dos dados	Falhas durante o pagamento podem gerar divergência entre o gateway e o banco.	Registrar o resultado da transação e tratar adequadamente respostas e erros do gateway; considerar reconciliação periódica.
Disponibilidade da localização	Indisponibilidade do serviço GPS impede o acompanhamento em tempo real.	Tratar falhas e indisponibilidade do serviço externo, exibindo última localização conhecida.
Escalabilidade do monólito	Crescimento do número de transportadoras pode exigir maior capacidade de processamento da API única.	Estrutura modular interna permite extrair módulos como serviços independentes no futuro, se necessário.
Denúncias sem fluxo de moderação detalhado	RF16 está descrito nos requisitos, mas ainda carece de um fluxo de execução e de regras de moderação (SLA de resposta, critérios de suspensão de posto).	Débito técnico a ser endereçado em uma próxima iteração da arquitetura.

12. Glossário

Termo	Definição
FuelWise	Plataforma para controle de combustível, pagamentos e gestão da frota.
Transportadora	Empresa cliente que utiliza o FuelWise para administrar sua frota e abastecimentos.
Caminhoneiro	Usuário operacional responsável por realizar abastecimentos durante as viagens.
Gerente	Usuário responsável pela administração da transportadora, frota, caminhoneiros e aprovação de gastos.
Administrador da Plataforma	Usuário responsável pela aprovação de transportadoras, gerenciamento de postos e tratamento de denúncias na plataforma.
Posto	Estabelecimento de combustível filiado ou consultável na plataforma, com preços e disponibilidade cadastrados.
Abastecimento	Operação de compra de combustível realizada durante uma viagem.
Viagem	Relação entre um caminhoneiro e um veículo da frota para uma operação.
Denúncia	Registro feito por um usuário sobre irregularidade em um posto ou em um pagamento, sujeito a verificação por um Administrador.
Gateway de Pagamento	Serviço externo responsável pelo processamento dos pagamentos.
JWT	Mecanismo (JSON Web Token) utilizado para autenticação e controle de acesso.
API	Interface utilizada para comunicação entre a aplicação e os serviços do FuelWise.